PYTORCH LIVE SESSION

Dataset, DataLoader & Augmentations

One image, four masks, a batch of useful variations

DLGenAI 26T3 / Live session 2

The session pipeline

Dataset prepares one image and its masks

The 80/20 split reserves a validation group

DataLoader combines samples into batches

Augmentation changes the training view

Which step decides what one sample contains?

Our teaching dataset

24

labelled images

96

annotation rows

4

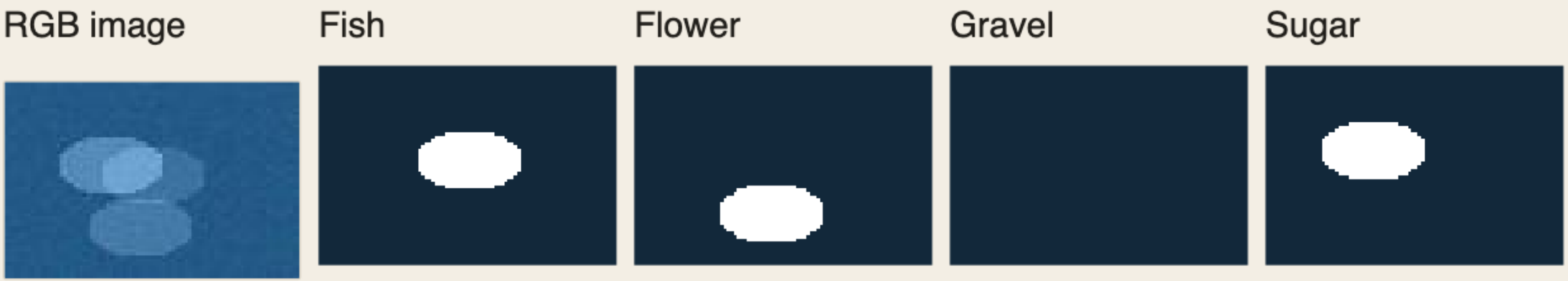
mask labels

Original synthetic cloud-like images

RGB images: 60 pixels high, 90 pixels wide

Independent teaching examples. Not real Kaggle cloud annotations.

One image has four answer sheets



Each label has its own binary mask: 0 = absent, 1 = present

Image and decoded masks from one original synthetic sample. White = 1, dark = 0.

Image channels and mask channels

Image

[3, 60, 90]

Red, green and blue

Target

[4, 60, 90]

One binary mask per cloud label

At one pixel, mask channels can contain [1, 0, 0, 1].

Separate binary channels allow overlapping labels. Background is [0, 0, 0, 0].

The annotation index

```
demo_001.jpg_Fish      RLE string
demo_001.jpg_Flower    RLE string
demo_001.jpg_Gravel    empty
demo_001.jpg_Sugar     RLE string
```

Group all four rows under one filename

Keep mask channels in a fixed label order

One CSV row or one image: which is our sample?

Three Dataset methods

```
dataset = CloudDataset(ROOT)  # __init__
len(dataset)                  # __len__
sample = dataset[1]           # __getitem__
```

Store the paths and annotation index once

Load one image when a sample is requested

dataset[1] requests the second indexed image. Images remain on disk until needed.

Inside __getitem__

```
filename = self.index.index[i]
image = open_image(filename)
masks = decode_four_masks(filename)
image, masks = resize_and_tensorize(image, masks)
return {"image": image, "mask": masks}
```

Decode RLE at the original image dimensions

Resize image and masks to matching dimensions

Conceptual pseudocode. The companion notebook contains the runnable implementation.

The tensor contract

```
image:  [H, W, 3]  becomes  [3, H, W]
masks:  four [H, W] arrays stack to [4, H, W]
```

Image intensities scale from 0-255 to 0-1

Binary mask values stay 0 or 1

An overlay checks spatial alignment

Can correct shapes hide an incorrect mask?

Why reserve validation images?

Training

Practice

Images and masks update model weights

Validation

Check

Different labelled images measure progress

Remembering training images can hide overfitting.

Validation guides choices. A separate test set evaluates the final chosen approach.

An 80/20 image split

19

training images

5

validation images

```
cut = int(0.8 * 24)  # 19
train_indices = order[:cut]
val_indices = order[cut:]
```

Whole-image rounding gives 19/24 and 5/24. 80/20 is a starting choice, not a universal rule.

Leakage through annotation rows

```
demo_001.jpg_Fish      training
demo_001.jpg_Flower    validation
```

Both rows refer to the same photograph

Keep every image and all its labels in one group

Keep related crops or frames together too

Would a flipped copy make this an independent image?

A repeatable split

```
g = torch.Generator().manual_seed(42)
order = torch.randperm(len(dataset), generator=g).tolist()
cut = int(0.8 * len(order))
train_ids, val_ids = order[:cut], order[cut:]
assert set(train_ids).isdisjoint(val_ids)
```

Keep membership fixed when comparing experiments

Audit rare-label coverage in both groups

A fixed seed repeats the ordering for the same setup. It does not guarantee balanced labels.

DataLoader builds the batch

```
loader = DataLoader(
    train_data, batch_size=4, shuffle=True,
    num_workers=0, drop_last=False,
)
batch = next(iter(loader))
```

Choose indices, fetch samples, stack matching tensors

Start with zero workers for notebook debugging

DataLoader does not train the model or automatically move tensors to a GPU.

One sample and one batch

Sample

[3, 60, 90]

Mask: [4, 60, 90]

Batch of 4

[4, 3, 60, 90]

Masks: [4, 4, 60, 90]

The first batch dimension counts images.

In a mask batch, the next dimension counts label channels. These two 4s mean different things.

Activity: the final batch

19 images. Batch size 6.

How many batches do we get?

How many images are in the final batch?

What changes with drop_last=True?

Explore the batch builder (click to open)

Augmentation changes a training view

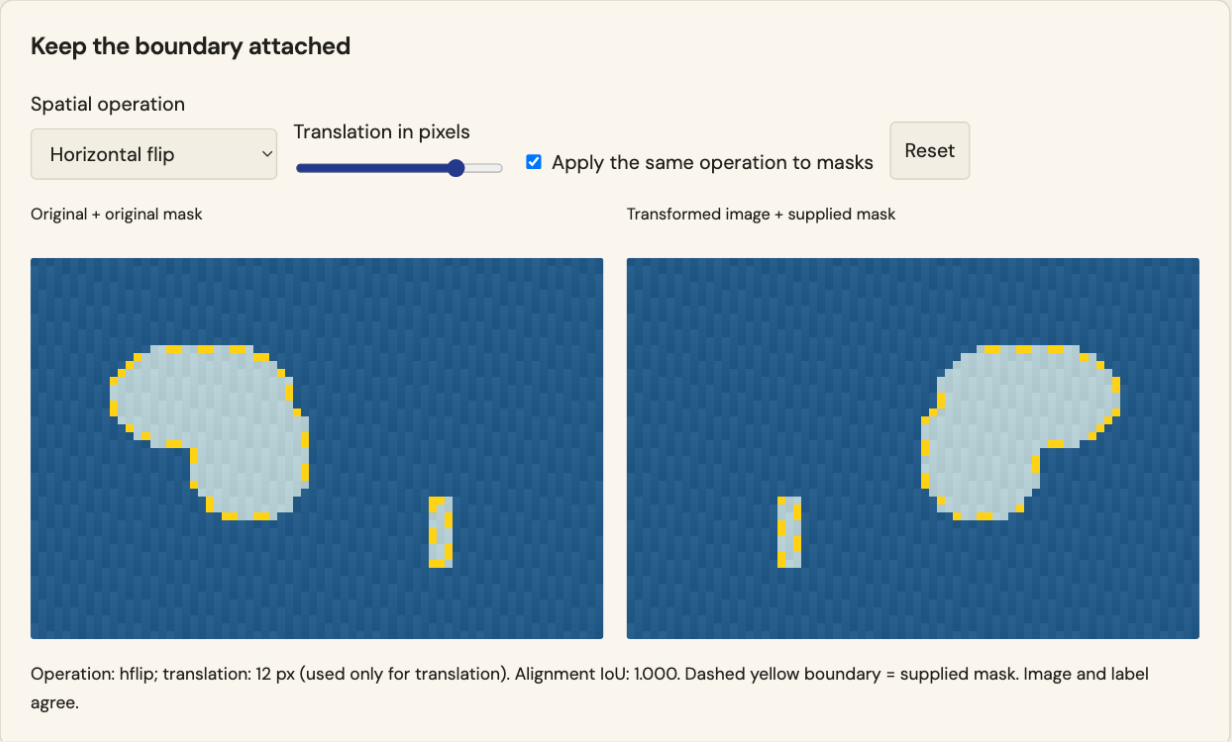
Geometry moves the image and every mask together

Appearance changes only the image

Random views do not increase Dataset length

Training gets random augmentation. Validation keeps deterministic preprocessing.

A paired horizontal flip



One shared geometric decision keeps the label attached.

Open the alignment lab (click to open)

Activity: a detached label



The image moved. The supplied mask stayed behind.

Would checking tensor shapes detect this bug?

Brightness and mask interpolation

Brightness multiplies image intensities

Mask values remain 0 or 1

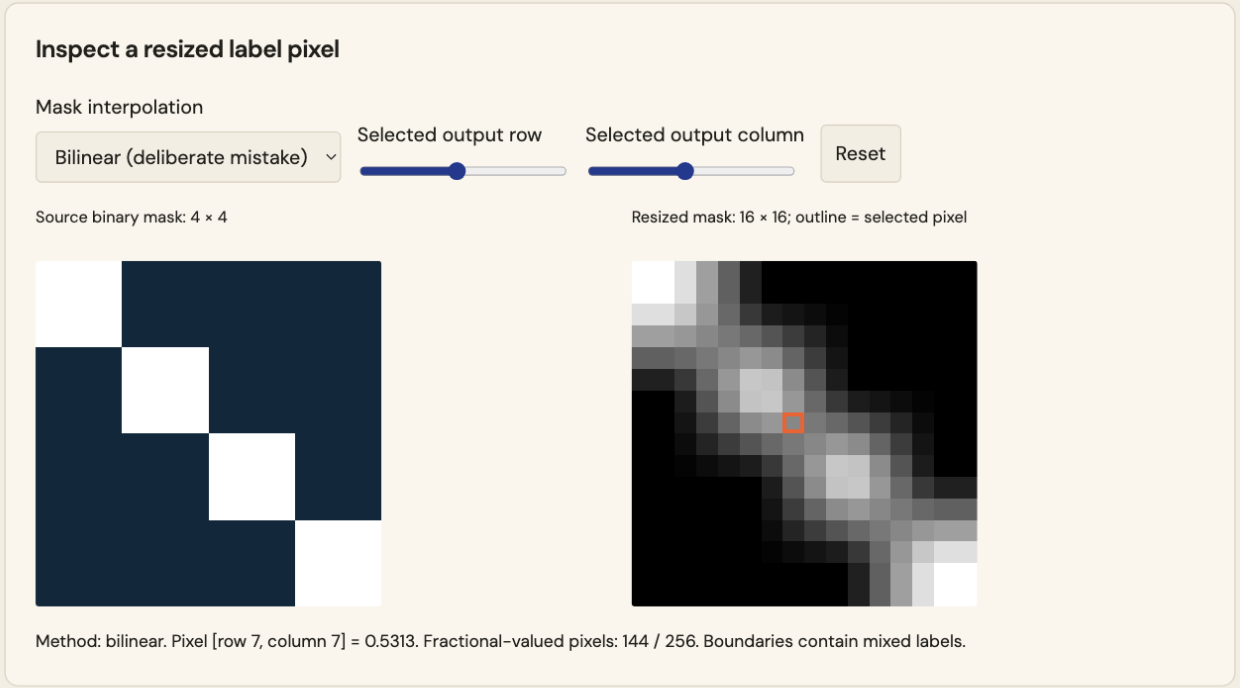
Bilinear interpolation suits image intensities

Nearest-neighbor preserves binary mask labels

Open the brightness and interpolation labs (click to open)

Nearest-neighbor can still lose thin structures during downsampling.

Activity: inspect a boundary pixel



Bilinear enlargement creates fractional mask values.

What changes when you switch to nearest-neighbor?

One random draw for both tensors

```
if torch.rand(()).item() < 0.5:
    image = torch.flip(image, dims=(-1,))
    mask = torch.flip(mask, dims=(-1,))

image = (image * brightness).clamp(0, 1)
```

Width is the last axis in C,H,W tensors

A 50% probability does not guarantee half the samples

Explore the probability lab (click to open)

Your explanation of the pipeline

What does dataset[5] return?

Why do all four labels stay in the same split?

Which dimension does DataLoader add?

Which transforms must also change the masks?

Open the notes and runnable notebook (click to open)

Created by manaykaistha. Source: Session 2 teaching notes and original synthetic lab screenshots.